SAILAB against other flood models: a controlled case study

Same terrain, rainfall, friction and scoring for every model, scored against the UNOSAT satellite flood map of the 2022 Sindh floods.

Generated 31 October 2026. SAILAB v6.1, LISFLOOD-FP 8.2.

1. What this shows, in plain terms

> **SAILAB's accuracy is on par with the other open 2D flood models, not better.** Across three test areas SAILAB's mean CSI is 0.504, against 0.503 (ANUGA), 0.497 (TELEMAC-2D) and 0.500 (SFINCS). The differences are about 0.01 to 0.03 and the three areas disagree on the ordering, so there is no winner on accuracy. SAILAB and ANUGA flooded almost the same cells (94% overlap), and TELEMAC-2D matched SAILAB at 90%. Independent codes agreeing this closely is good evidence that SAILAB's solver is correct.

> **No model, including SAILAB, scored clearly better than the trivial "flood everything" guess** (mean CSI 0.500). Raising the wet-depth cut-off makes every model score worse (section 5). The most likely reason is the forcing, not the models: the 2022 Sindh flood was mostly river overflow and upstream water, and this test applies only local rainfall. This has not been tested directly.

Where SAILAB does have a measured advantage:

- **Speed.** On the same cases SAILAB's GPU solver took 58 s on average, against 141 s (TELEMAC-2D), 528 s (SFINCS) and 477 s (ANUGA). The comparison is GPU against CPU, ANUGA and TELEMAC used a coarser 90 m mesh, and TELEMAC ran on one core, so it shows what a user gets, not a like-for-like solver speed test.
- **Integration and cost.** SAILAB runs the whole workflow (rainfall, buildings, roads, loss, scenarios, monitoring) on one machine with no licence fee. None of the other models do that on their own.
- **Portability.** SAILAB's solver runs on this ARM GPU server; the commercial and Windows tools in section 7 could not be run here at all.

Where it is not ahead: accuracy (parity), depth validation (none for any model here), and the volume of published validation behind HEC-RAS, TUFLOW, MIKE and others.

2. Study design

- **Event and observed map:** the 2022 Pakistan floods, Sindh. Observed flood extent from UNOSAT (VIIRS satellite, 29 November 2022). The map is from three months after the August peak, so it shows water that persisted, not the rain-driven peak.

- **Areas:** three 13 km boxes placed along the edge of the observed flood so each is about half flooded (49%, 50%, 51%). An earlier test box that was 98% flooded was dropped because it cannot show skill.
- **Same inputs for every model:** SRTM elevation (29 m), 253.5 mm of rain over 48 hours (the wettest 48-hour ERA5 window at the site), Manning n = 0.05, no infiltration, closed boundaries, nothing calibrated.
- **Scoring:** cells counted wet above 5 cm peak depth. CSI = hits / (hits + misses + false alarms); POD = share of observed flooded cells found; FAR = share of predicted wet cells that were dry in the observation.
- **Baselines:** "Flood everything", and "lowest terrain", which floods the lowest cells up to the true flooded fraction (it is told the answer's size).
- **Models run:** SAILAB (LISFLOOD-FP 8.2, ACC solver, GPU); ANUGA 4.1 (full shallow-water, finite volume, 90 m mesh); SFINCS (Deltares, local-inertial, built from source); TELEMAC-2D (full shallow-water, finite element, built from source, 90 m mesh, constant rain at the same total).

3. Results

Model	CSI A	CSI B	CSI C	Mean CSI	POD	FAR	Wet area / observed	Mean run time	Hardware
Flood everything	0.486	0.505	0.509	**0.500**	1.00	0.50	2.00x	-	-
Lowest terrain (oracle fraction)	0.392	0.507	0.347	**0.415**	0.58	0.42	1.00x	-	-
SAILAB	0.504	0.528	0.481	**0.504**	0.89	0.46	1.65x	58 s	GPU (NVIDIA GB10)
ANUGA	0.495	0.535	0.478	**0.503**	0.86	0.45	1.57x	477 s	CPU, 90 m mesh
SFINCS	0.486	0.505	0.509	**0.500**	1.00	0.50	2.00x	528 s	CPU, 20 threads, 29 m
TELEMAC-2D	0.490	0.531	0.469	**0.497**	0.84	0.45	1.53x	141 s	CPU, 1 core, 90 m mesh

Areas: A Manchar edge, B north Dadu, C Shikarpur. Italic rows are the trivial baselines. Wet area / observed above 1 means over-prediction.

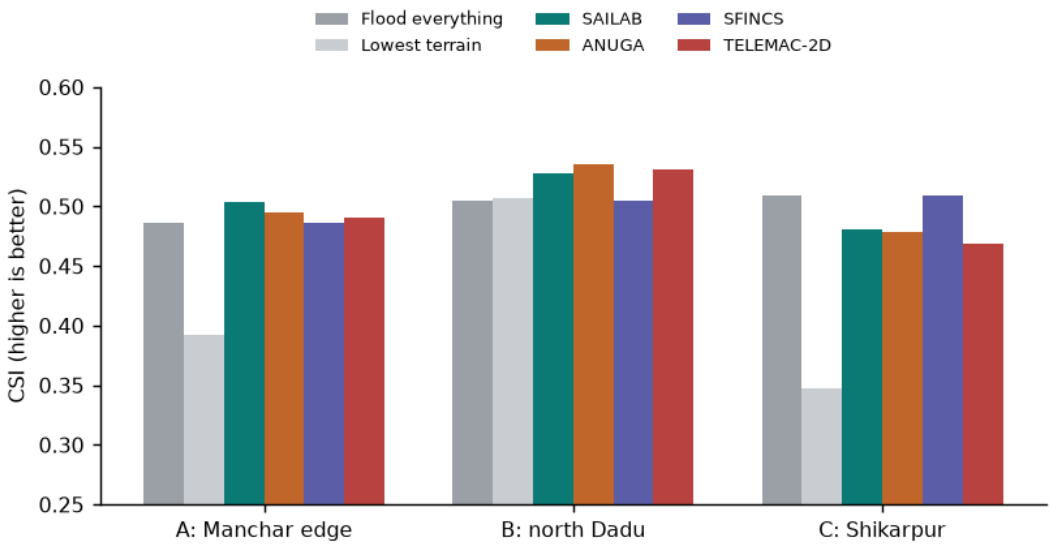


Figure 1. CSI by area. The grey bars are the baselines.

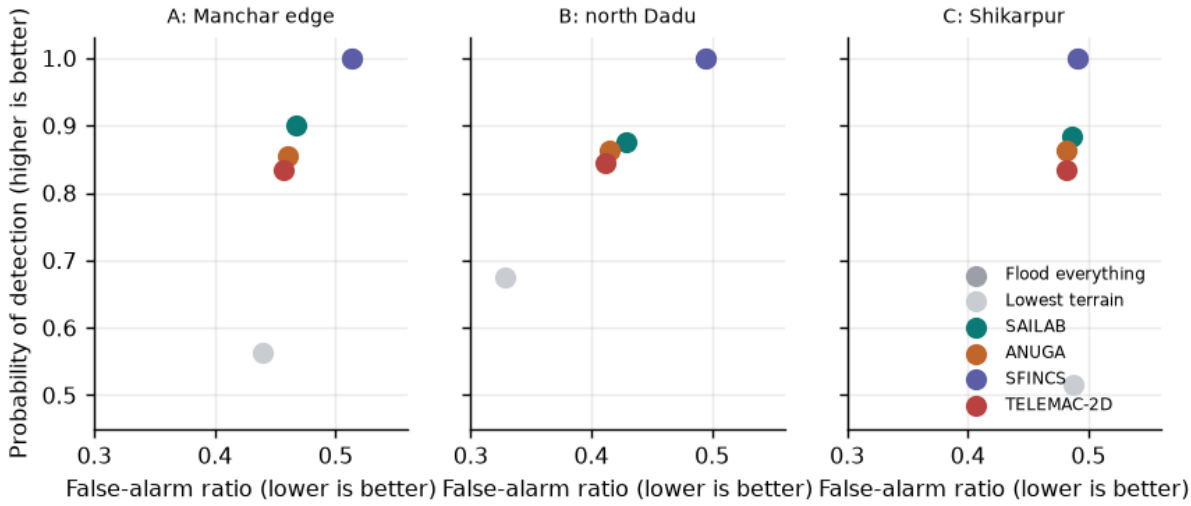
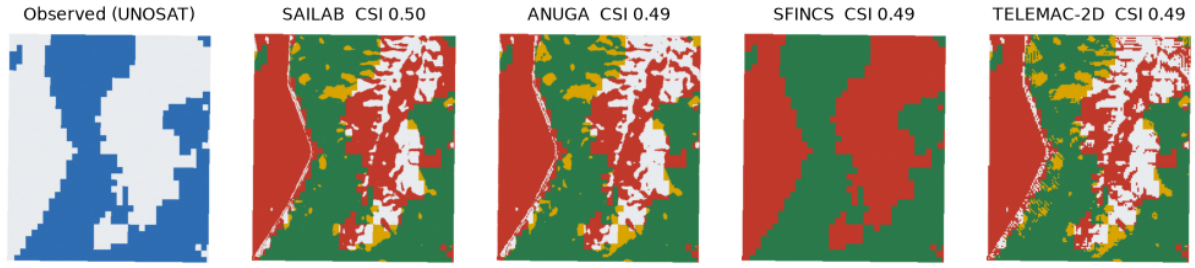


Figure 2. Detection against false alarms. All four models find 83 to 100% of the flooded cells; SFINCS reaches 100% only because it reports more than 5 cm of water in every cell.

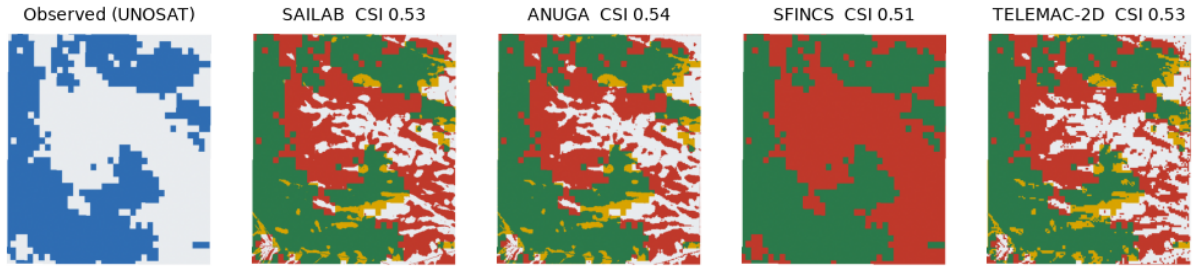
4. Maps

Key: correctly wet missed false alarm correctly dry

Area A: Manchar edge



Area B: north Dadu



Area C: Shikarpur

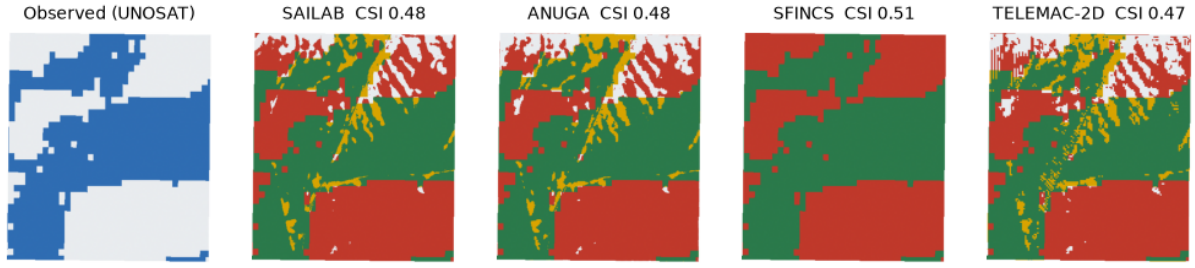


Figure 3. Observed flooding (left, blue) and each model's result against it. All four models flood the low western strip of area A, which the satellite map shows as dry.

Peak depth, area A

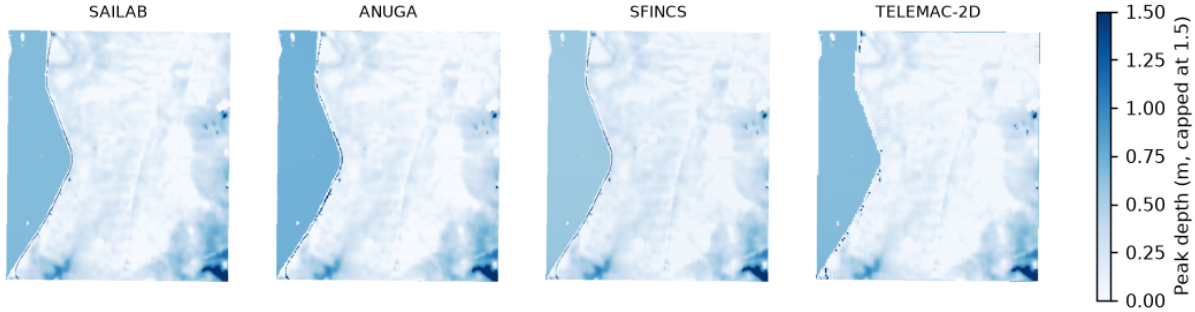


Figure 4. Peak depth from each model (same scale). The patterns are alike; maximum depths are 4.1 to 4.5 m.

5. How robust is the result

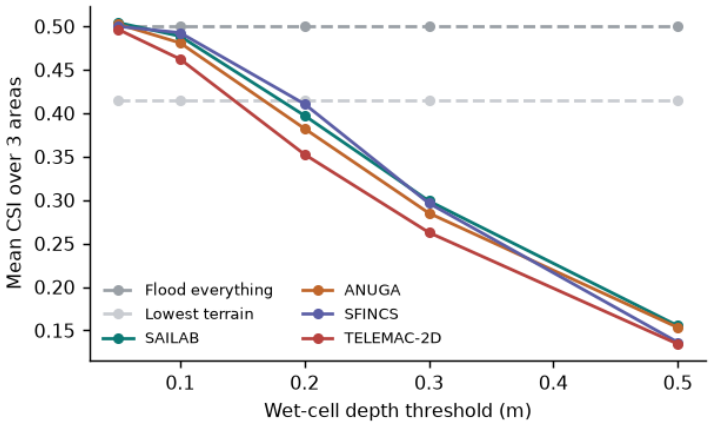


Figure 5. Mean CSI as the wet-depth cut-off rises. Every model falls below "flood everything" at every cut-off from 10 cm up, and below both baselines from 20 cm up (SAILAB beats "flood everything" only at 5 cm, by 0.004), so the models' depth patterns do not line up with the observed extent. A model that is right for the right reasons would hold up better as the cut-off rises.

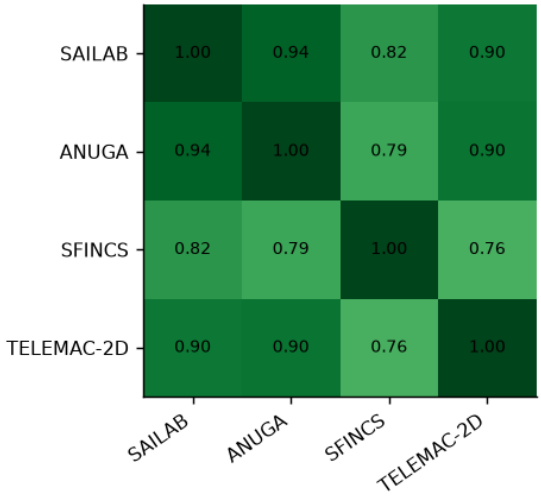


Figure 6. How much the models' flooded cells overlap each other (1 = identical). SAILAB, ANUGA and TELEMAC-2D agree at 0.90 to 0.94; SFINCS differs most because it keeps a thin layer of water everywhere.

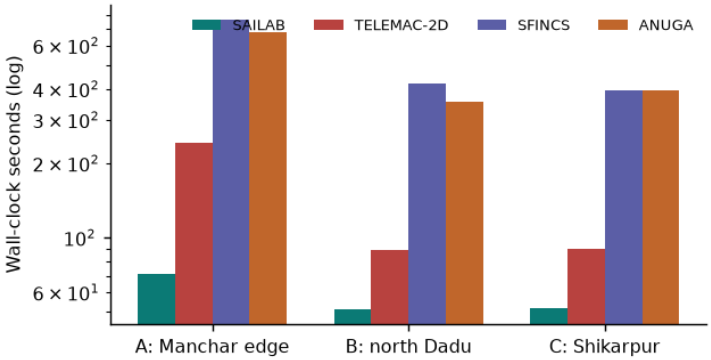


Figure 7. Run times (log scale). Hardware differs between models, see section 1.

6. What the evidence supports

- SAILAB's solver gives the same flood as independent codes on the same inputs: this is the strongest finding, and it supports the correctness of the numerics.
- It cannot yet claim better real-world accuracy than the other models, and this test alone cannot show whether any of them has real skill on this flood.
- SAILAB ran about 2 to 9 times faster than the CPU models on these cases (TELEMAC-2D 2.4x, ANUGA 8x, SFINCS 9x on average), with the hardware caveat above.

> **Limits of this study.** One event; three small areas; observed map three months after the peak; rain-only forcing that cannot represent river overflow; no depth data to compare against; meshes of 29 m and 90 m differ between models; each model's own numerical settings were left at defaults; the area-A west strip may reflect SRTM elevation error. None of these favour or disadvantage SAILAB specifically.

7. Models not run, and why

Model	Why it was not run	What that means for SAILAB
HEC-RAS 2D	Only Windows and x86 Linux builds exist; this server is ARM. A 2D case also needs a mesh/ geometry file normally built in its Windows GUI.	Free, but cannot run here. SAILAB runs on this machine with no licence or Windows.
TUFLOW (Classic/HPC/FV)	Commercial licence required, Windows/x86.	No licence cost for SAILAB's solver (GPL).
MIKE 21 / MIKE+	Commercial licence (DHI), Windows.	Same.
InfoWorks ICM	Commercial licence (Autodesk), Windows.	Same.
Flood Modeller	Commercial licence, Windows.	Same.
FLO-2D	Windows desktop program; Pro tier is paid.	Same.
Delft3D-FM / D-Flow FM	Official binaries are x86; no ARM package found.	SAILAB is built from source for this ARM/GPU machine.
TRITON, SERGHEI, RIM2D, HiPIMS	Need source builds; GPU support on this ARM GPU is untested. Not attempted for time.	Not a SAILAB advantage: untested.
Fathom, Google Flood Hub, GloFAS	Data products or services, not runnable models.	Published scores only (see survey).

Not running a model is not evidence that it is worse or better than SAILAB. What it does show is a practical difference: SAILAB's solver is open and runs on this ARM GPU server without a licence or Windows, while most of the established tools need a licence, Windows or x86 hardware. For these models the survey (SAILAB_Flood_Model_Survey.md) lists published validation results instead; HEC-RAS, for example, has been benchmarked against

LISFLOOD-FP (the engine inside SAILAB) on the Secchia River flood, where both mis-estimated extent and depth in built-up areas.

8. Recommended next steps to make the accuracy case

1. Score an event where the flood can be reproduced from its drivers: known river inflow (for example the 2010 Indus flood with gauge hydrographs), or a rain-driven urban flood with a mapped extent.
2. Add observed depths (high-water marks, gauges) so depth can be scored, not just extent.
3. Run SAILAB with river inflow from GloFAS on the 2022 areas, to test the river-overflow explanation.
4. Run HEC-RAS or TUFLOW on the same three areas on a licensed x86 machine, and score with the same code (inputs are available).
5. Try 8.3 of LISFLOOD-FP on a multi-million-cell basin; on our test cases it matched 8.2 and showed no speed gain.

9. Reproducibility

All code and inputs are in `~/flood/cmp`: `case_common.py` (shared DEM, observed map and scoring), `run_baselines_sailab.py`, `run_anuga.py`, `run_sfincs.py`, `run_telemac.py`, `score_all.py`, `report_build.py`. Raw numbers: `results_all.json`, `results_extra.json`. Models: SAILAB/LISFLOOD-FP 8.2; ANUGA 4.1.0 (pip); SFINCS v2.5.0-beta (Deltares, built from source 31 Oct 2026); TELEMAC v9.2-dev (built from source, serial). The first SFINCS attempt used a conda package with the same name that turned out to be a plasma-physics code; it was removed and no result came from it. The earlier SAILAB test reporting CSI 0.66 used a box that was 98% flooded and should not be quoted as accuracy.